

Demand Forecasting

An end-to-end demand-forecasting study on the Walmart M5 retail dataset

Summary

Every inventory decision a retailer makes is a bet on the future. Order too much and you tie up cash, fill shelves with product that ages, and eventually mark it down. Order too little and you miss the sale, frustrate the customer, and quietly train them to shop somewhere else. The whole job of demand science is to make that bet a little less of a gamble and to be honest about how much of the outcome we can actually control.

This Project is broken into two parts, around the idea of using the M5 dataset (five and a half years of daily Walmart sales across California, Texas and Wisconsin). The first project answers “how much will we sell?” It is a full forecasting pipeline, from raw data to a model that beats the standard retail benchmark and produces the kind of probabilistic forecast you can actually set safety stock against. The second project answers a different question: “if we change something, how much does demand move because of our change, and not because of everything else going on?” For that I used SNAP food-assistance days as a natural experiment and triangulated the causal effect three different ways.

Together they cover the two halves of the demand-science job: predict the baseline, then measure the levers. The rest of this document walks through the business problem, what I did, what I found, and what I would do next if this were a real product.

The business problem

Picture a single store on a single Tuesday. It stocks roughly three thousand different products, and for most of them the honest answer to “how many will sell today?” is “probably zero, maybe one.” A handful of fast movers carry the volume; few items barely sell at all. Now multiply that by ten stores and then by every day of the year, and you have the planning problem a supply-chain team lives with.

What makes it matter is that the cost of being wrong is disproportionate. A stockout and an overstock are not equal and opposite mistakes. A stockout is a lost sale plus a small dent in trust that you never see on a dashboard. An overstock is holding cost, markdowns, and sometimes spoilage. Because the two failures cost different amounts, the “best” forecast is usually not the one that nails the average, but it is the one that respects the asymmetry and leans toward whichever error is cheaper for that product. That single idea shapes a lot of the choices later in this write-up.

Why this matters

Retail demand breaks most of the assumptions that make forecasting tidy. It is intermittent: a large share of products sell nothing on most days, so the usual error metrics quietly fall apart because you cannot compute a percentage error against a day with zero sales. It is non-stationary: there is a clear upward trend as the business grows, a strong weekly rhythm, and an annual cycle on top. It is event-driven: holidays, promotions, price changes and benefit-program days all push demand around, and they often happen at the same time, which makes their individual effects hard to separate. And a zero in the data is ambiguous as it can mean “nobody wanted it” or “we were out of stock” or “the product wasn’t even on the shelf yet,” and those three things should be treated completely differently.

I mention all of this up front because most of the work in a credible forecasting project is not the model. It is noticing these traps and handling them deliberately.

The data we worked with

The M5 dataset is the public Walmart sales data released for the M5 forecasting competition. It contains 30,490 daily time series spanning January 2011 through June 2016 — three states, ten stores, three categories (foods, household, hobbies), and 3,049 individual products. Alongside the sales it ships a calendar of holidays and events, weekly selling prices for every product, and flags marking SNAP days (the days when U.S. food-assistance benefits can be spent). That SNAP information turns out to be the key that unlocks the second project.

To run this locally, I built the deep modeling pipeline on one store (California store 1, all 3,049 products).

Project 1: Forecasting - How much we will sell?

Building a foundation we can trust

The first notebook is unglamorous on purpose. I profiled every raw table, shrank them with sensible data types so the work fits in memory, and reshaped the wide sales matrix into a tidy day-by-day panel joined to the calendar and price tables. The decision I care most about here is how I handled zeros. I separated three cases: the period before a product’s very first sale (it wasn’t listed yet, so those zeros are noise and I dropped them), genuine zero-demand days inside a product’s active life (real signal, kept), and weeks where the price was missing, which usually means the item was out of stock or not offered (kept, but flagged). Getting this wrong is one of the most common ways a retail forecast picks up a hidden bias, so it was worth doing carefully.

I also flagged outliers, but I did not delete them. In retail a spike is often a real promotional surge and it is exactly the demand we most want to predict, so I used a robust, per-product filter that catches implausible data-entry errors while leaving genuine volatility alone.

Getting to know the data

Before touching a model I spent real time understanding the series. A demand-classification view showed that a large fraction of products are intermittent or lumpy rather than smooth, which immediately rules some classic techniques out and tells you which error metrics will lie to you. Seasonal decomposition confirmed the strong weekly cycle and the upward trend; formal stationarity tests (ADF and KPSS together, because they disagree in useful ways) told me exactly how much differencing the classical models would need rather than leaving me to guess; and the autocorrelation plots pointed straight at weekly memory at lags of 7, 14 and 28 days, which is what I leaned on when building features.

The driver analysis is where the business story started to show up. Food demand on SNAP days runs noticeably higher than on ordinary days, while hobbies barely budge which makes sense, because food assistance buys groceries, not toys. Event days shift demand, and most products show the textbook negative price response (raise the price, sell a little less). Those three signals such as recent sales history, price, and SNAP became the backbone of the model, and reassuringly they are exactly what the model later told me it relied on most.

Choosing and testing the models

I deliberately built up from simple to complex so that every step had to earn its place. The bar to beat is the seasonal-naive forecast, “next Tuesday will look like last Tuesday” which is a surprisingly strong competitor when the weekly rhythm is this dominant. I then fit classical statistical models (exponential smoothing and SARIMA) on the aggregated series, and finally a single gradient-boosted model (LightGBM) trained across all products at once, fed the lag, price, SNAP and calendar features.

Two choices are worth calling out. I trained the machine-learning model with a Tweedie loss, which is built for demand that is mostly zeros with a long positive tail and ordinary squared-error loss would have biased it. And I validated everything with an expanding-window scheme that always trains on the past and tests on the future, because the single fastest way to fool yourself in forecasting is to let the model peek across time. I also ran a small complexity sweep to show the bias–variance trade-off honestly: as I added capacity, training error kept dropping while validation error barely moved, which is the model fitting noise rather than signal.

What I found

The global LightGBM model came out ahead, and it did so while forecasting every product at once, which is the version of the problem a real team has to solve. The table below uses RMSSE, the competition’s scale-free accuracy metric, where anything under 1.0 means “better than seasonal-naive.”

Model	RMSSE (lower is better)	Read
LightGBM (global ML)	0.595	Best; one model for all products

Exponential smoothing	0.636	Strong classical baseline
SARIMA	0.649	Orders chosen from the stationarity tests
Seasonal-naive	0.681	The bar everything must clear
Naive	2.676	Last value carried forward; far behind

Across the expanding-window folds the model scored 0.566 on average with a standard deviation of about 0.09, so the performance is stable, not a lucky split. But the result I would actually put in front of a planner is not the point forecast at all but it is the probabilistic one. I trained the model to predict the 95th percentile of demand, and when I checked it against reality it covered the actual sales 94.5% of the time. That number is directly useful: the 95th-percentile forecast is, in plain terms, the stock level that keeps you in stock about nineteen days out of twenty. That is the bridge from a data-science output to an inventory decision, and it is where the asymmetry I mentioned at the start finally gets handled properly.

Project 2: Measuring causal effect – The effect of something we change

A different kind of question

A forecast tells you what is likely to happen if nothing changes. It does not tell you what happens if you do something like drop a price, speed up replenishment, run a promotion. Those are causal questions, and you cannot answer them by noticing that two things move together. SNAP days are busy for groceries, but SNAP days also cluster at the start of the month and fall on particular weekdays, and those things drive grocery shopping on their own. So the raw comparison is contaminated. The job of this project was to separate the part of the lift that SNAP actually causes from the part that just happens to coincide with it.

Using SNAP days as a natural experiment

SNAP is close to an ideal natural experiment for this. It is scheduled by policy rather than by Walmart's marketing, so it is not tangled up with the store's own promotions. It should move food but not hobbies, which gives me a built-in control group. And the schedule differs by state, which makes a proper before-and-after comparison possible. I leaned on all three of those properties.

Separating cause from coincidence

I approached it from three angles, because any one of them alone is easy to argue with. First, the naive comparison, stated honestly as the biased starting point: food sales run about 17% higher on SNAP days. Second, a placebo check, the same comparison for hobbies, which barely

moves, telling me the effect is genuinely about food and not just “SNAP days are busy days.” Third, a difference-in-differences model that uses hobbies as the control and strips out day-of-week and seasonal effects shared by both categories. That last one is the estimate I trust most.

Method	Estimated SNAP effect on food	What it controls for
Naive comparison	+17.3%	Nothing — confounded, shown for contrast
Placebo (hobbies)	+2.5%	Sanity check; should be near zero
Difference-in-differences	+15.1% (95% CI 11.4–18.9%)	Shared weekday/seasonal/event effects
Propensity weighting (IPW)	≈ unchanged from naive	Calendar covariates within food only

There is an honest subtlety in that last row. The propensity-weighting method barely changed the food-only estimate and rather than hide that, it is worth explaining, because it is informative. SNAP runs on a fixed monthly schedule, so within food its timing is close to random with respect to the weekday and seasonal factors I could measure; there simply wasn’t much measurable confounding left to remove at that level. The real correction came from the difference-in-differences view, which pulled the estimate from 17% down to about 15% by removing the effects food and hobbies share. The point is that three methods built on different assumptions all land in the same neighborhood and tell the same story. That agreement is what earns trust in a causal claim.

And how I would test it for real

Observational methods can only correct for the confounders you can see. The clean way to settle a causal question is a randomized experiment, so I sketched one for a concrete supply-chain lever: a faster replenishment policy meant to cut out-of-stocks. I would randomize at the store level, stratified by state and baseline volume, measure realized sales per store-day with holding cost as a guardrail, and run a difference-in-means with a pre-period adjustment to tighten the estimate. A power calculation says that to reliably detect a 3% lift I would need roughly 1,600 store-days per arm which, importantly, tells leadership up front how long the test must run to give a trustworthy answer rather than an inconclusive one.

What this means for the business

Put the two projects side by side and you get a small but complete decision loop. The forecasting model gives a planner a defensible baseline and, more usefully, a probabilistic target

that respects the real cost of running out versus over-ordering. The causal work then says which levers are worth pulling and how hard: SNAP days really do lift food demand by a meaningful, repeatable amount, which means that demand should be pre-positioned as inventory and staffing rather than chased after the fact. One project predicts the world; the other measures what we can change about it. A demand team needs both, and they need to talk to each other.

What I would do next

If this were heading toward production, my next steps would be, roughly in order: scale the pipeline to all ten stores and reconcile forecasts up the product hierarchy so store-level numbers add up to state-level ones; add the proper competition-weighted accuracy metric across the full hierarchy and only then consider a deep-learning model such as a Temporal Fusion Transformer, which earns its complexity when you have many dense series and rich covariates and can output quantiles directly. I deliberately did not reach for deep learning here as the gradient-boosted model delivers most of the value at a fraction of the operational cost, and knowing when not to use the fancy tool is part of the job. On the causal side, I would move from the natural experiment to the randomized one I scoped and build a small monitoring layer so the forecast's accuracy and bias are tracked over time rather than assumed.

Appendix A : Results

Dataset: 30,490 daily series · 3 states · 10 stores · 3,049 products · Jan 2011 – Jun 2016. Deep pipeline run on store CA_1 (all products); forecast horizon 28 days.

Forecast accuracy (RMSSE, lower is better; <1 beats seasonal-naïve):

LightGBM 0.595 · Exponential smoothing 0.636 · SARIMA 0.649 · Seasonal-naïve 0.681 · Naïve 2.676

Cross-validated RMSSE (expanding window): 0.566 ± 0.094

Probabilistic forecast: 95th-percentile model achieved 94.5% empirical coverage i.e., it kept stock sufficient about 19 days out of 20.

Hierarchy-weighted accuracy (wRMSSE): 0.770 across the store's products.

Causal effect of SNAP days on food demand:

Naïve (biased) +17.3% → difference-in-differences +15.1% (95% CI 11.4–18.9%) · hobbies placebo +2.5% (near zero, as expected).

Experiment design: store-level randomization; ~1,600 store-days per arm needed to detect a 3% lift at 80% power.